

KIRK Ltd.

SALINA

Software Requirements Specification

Version 0.01

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: <i>03/17/2026</i>
<i>SALINA</i> v.0.01	

Revision History

Date	Version	Description	Author
03/17/2026	v0.01	Commencement of system design, including multi-tenant architecture mapping, UI/UX specifications, and database schema formulation.	K.I.R.K Ltd.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Table of Contents

- Introduction..... 5**
 - Purpose..... 5
 - Scope..... 5
 - SaaS Strategy..... 6
 - TARGET AUDIENCE..... 6
 - TENANT MODEL..... 6
 - MONETIZATION INTERFACE..... 6
 - System Overview..... 6
 - Reference Material..... 6
 - Definitions and Acronyms..... 6
- System & Multi-tenancy Architecture..... 7**
 - Architectural Pattern..... 7
 - Data Isolation..... 7
 - Application Isolation..... 7
 - Atomic Infrastructure..... 8
 - Global Atoms..... 8
 - Tenant Molecules..... 8
 - Tenant Organisms..... 8
 - Templates/Pages..... 8
 - Authentication & RBAC (Role-Based Access Control)..... 8
 - Data Flow & Integration..... 9
 - Error Handling & Logging Strategy..... 10
 - Deployment & Environments..... 10
- Design System & UI/UX (Foundational Elements)..... 11**
 - Design Tokens (The Atoms)..... 11
 - Color Palette..... 11
 - Typography..... 11
 - Spacing and Grid..... 12
 - Desktop Grid Specifications..... 12
 - Desktop Spacing Scale..... 12
 - Desktop Application in Wireframes..... 12
 - Iconography..... 12
 - Theming Atoms (Design Tokens)..... 12
 - Component Library (Atoms & Molecules)..... 13
 - Atoms..... 13
 - Molecules..... 14
 - Search bars (Input + Button)..... 14
 - Card Headers..... 14
 - Form Groups..... 14

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Design Rationale & Accessibility.....	14
SaaS Experience (UX).....	14
Accessibility (a11y):.....	14
Atomic System Design.....	15
Provisioning Organism (The Onboarding Engine).....	15
Subscription & Tiering (Feature Flagging).....	15
Component Decomposition (The Organisms).....	16
Organisms 1: Navigation Sidebar.....	16
Organism 2: Data Dashboard Grid.....	16
Templates & Page Layouts.....	16
Wireframes/Screen Images.....	16
Data Design.....	16
Multi-tenant Schema Logic.....	16
Salina Entity-Relationship Diagram.....	17
Global Platform Tables (Non-Tenant Scoped).....	18
Tenant-Scoped Tables (Enforced by RLS).....	18
API Design & Rate Limiting.....	23
Tenant Scoping.....	23
Quotas.....	23
Database Backups & Disaster Recovery.....	24
Requirements Traceability Matrix.....	25
Appendices.....	26

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Software Requirements Specification

Introduction

Purpose

This document defines the architectural and technical specification for SALINA. It translates the functional and non-functional requirements into concrete design decisions, system blueprints, and implementation guidelines.

This Software Design Document serves as the authoritative reference and development blueprint for all engineering work on the SALINA system. It encompasses the system's multi-tenant architecture, data model, API layer, security control, and component hierarchy. All development activities shall be aligned with and traceable to the specification defined herein.

This document is intended for use by the SALINA development team, including frontend engineers, backend engineers, database architects and the leads of K.I.R.K Ltd.

Scope

IN-SCOPE ITEMS

1. The platform centralizes recruitment, data management, and internal communications.
2. The system provides a Policy Engine to configure recruitment workflows.
3. The architecture enforces database-level multi-tenancy.
4. The application features digital QR identification and event check-in attendance tracking for members.
5. Members can view organization-specific internal newsfeed along with ability to monitor dues and membership status.

OUT-OF-SCOPE ITEMS

1. Direct financial transactions and payment processing are excluded from the platform.
2. Users upload payment receipts for manual verification.
3. The system excludes academic grading or university-level enrollments.
4. The architecture excludes public-facing social media integrations.
5. Content remains restricted to the internal organizational feed and not published externally.
6. Native communication channels such as direct messaging, group chats, and video calling are excluded.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

SaaS Strategy

TARGET AUDIENCE

The platform targets any organization seeking to centralize recruitment, membership management, and internal operations. Organization Officers configure and manage the environment while Members and Applicants interact with the portal and pipeline.

TENANT MODEL

SALINA operates on a single shared backend where all tenants share the same Next.js deployment and Supabase PostgreSQL instance. Isolation is enforced at the database level through Row Level Security (RLS), which validates every query against the user's JWT `org_id` claim, ensuring no user can access another organization's data under any circumstance. Organizations enter the platform through a Super Admin accreditation flow, and only those granted Live status may onboard members and operate a recruitment pipeline. Each tenant independently configures its own brand identity such as logo, colors, and font and which is applied platform-wide through a white-labeled interface.

MONETIZATION INTERFACE

SALINA uses a single-tier access model where all accredited organizations receive the full feature set with no subscription tiers, feature flags, or paywalled capabilities. The platform excludes all financial transaction processing — dues statuses are tracked as member metadata and updated manually by Officers upon receipt verification. Interface design and feature availability remain consistent across all tenants.

System Overview

SALINA is built on a client-server architecture where a Next.js App Router frontend communicates directly with a Supabase PostgreSQL backend through Next.js Server Actions. This design eliminates the need for a separately deployed middleware or REST API layer, consolidating the application logic, data communication, and database state management within a single cohesive technical stack. The architecture is designed to support multi-tenant operations, enforcing organization-level data isolation at every layer from the frontend session down to the database query.

Reference Material

- SALINA Software Requirements Specification (v0.03)
- SALINA Project Approval Sheet (Submitted March 3, 2026)
- Cebu Institute of Technology - University SDD Guidelines

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Definitions and Acronyms

Tenant: A registered organization operating on the platform.

Policy Engine: The core logic system that dynamically alters the application workflow based on an organization's configuration.

RLS: A database-level security feature that enforces strict data isolation between tenants at the PostgreSQL layer.

Pipeline Stage: A specific step in a recruitment workflow as defined by the tenant's active policy configuration.

System & Multi-tenancy Architecture

Salina is a specialized, multi-tenant Software-as-a-Service (SaaS) engine designed to centralize recruitment, data management, and internal communications into one platform. Acting as an "Operating System" for organizations, it replaces disjointed workflows—such as scattered spreadsheets and chat groups—with a unified, configurable solution. The system enables multiple organizations to operate under a single shared backend while securely managing their entire lifecycle, from applicant tracking to active officer operations. This architecture is built upon a Next.js client-server model paired with a Supabase PostgreSQL backend, ensuring high-volume query performance and strict data security.

Architectural Pattern

The project utilizes a modern, serverless Client-Server architecture built on the Next.js App Router. The frontend leverages React Server Components to pre-render dynamic, multi-tenant UI layouts on the server, while Client Components handle interactive states. This architecture eliminates the need for a traditional standalone middleware layer (like Node.js/Express) by communicating directly with the cloud-hosted Supabase PostgreSQL backend via Next.js Server Actions.

Tenant Isolation Strategy

Because all organizations share a single database schema, strict tenant isolation is the highest security priority.

Data Isolation

Salina implements a **Shared-Schema with tenant_id filters** strategy, mathematically enforced at the PostgreSQL level using Supabase Row Level Security (RLS). Every database query automatically validates the user's active JSON Web Token (JWT) against the `org_id` column of the queried table (e.g., `auth.jwt() -> 'org_id' = organization_id`). This guarantees that cross-tenant data leakage is impossible.

Application Isolation

The application utilizes a **Shared runtime with namespace isolation**. By extracting the `org_id` from the URL slug or active session, the Next.js frontend dynamically swaps the organizational context (such as branding colors, typography, and policy JSONs) to provide the illusion of a dedicated,

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

standalone application for each tenant.

Atomic Infrastructure

The system infrastructure is partitioned following an adapted Atomic Design methodology, scaling from global services down to tenant-specific interfaces:

Global Atoms

These are the foundational, immutable services that power the entire platform. Key atoms include Supabase Auth for managing secure user sessions and JWT issuance, and Supabase Storage serving as the global bucket for hosting applicant documents and avatars.

Tenant Molecules

These are the configurable logic blocks injected into the system by individual tenants. Examples include the `theme_config` (which overrides global CSS variables for white-labeling personalization) and the `pipeline_config` (the JSON array dictating custom recruitment stages).

Tenant Organisms

These are complex, interactive modules driven by the active Molecules. Examples include the Real-time Internal Newsfeed powered by Supabase Realtime, and the dynamic Applicant Kanban Board restricted by the tenant's active pipeline stages.

Templates/Pages

The final layout wrappers (e.g., Next.js Route Groups) that assemble the Organisms, inject the Tenant Molecules (colors/fonts), and serve the fully branded, authenticated experience to the end-user.

Authentication & RBAC (Role-Based Access Control)

All user identities are authenticated via Supabase Auth using secure JSON Web Tokens. The platform implements a dynamic, multi-tier RBAC system:

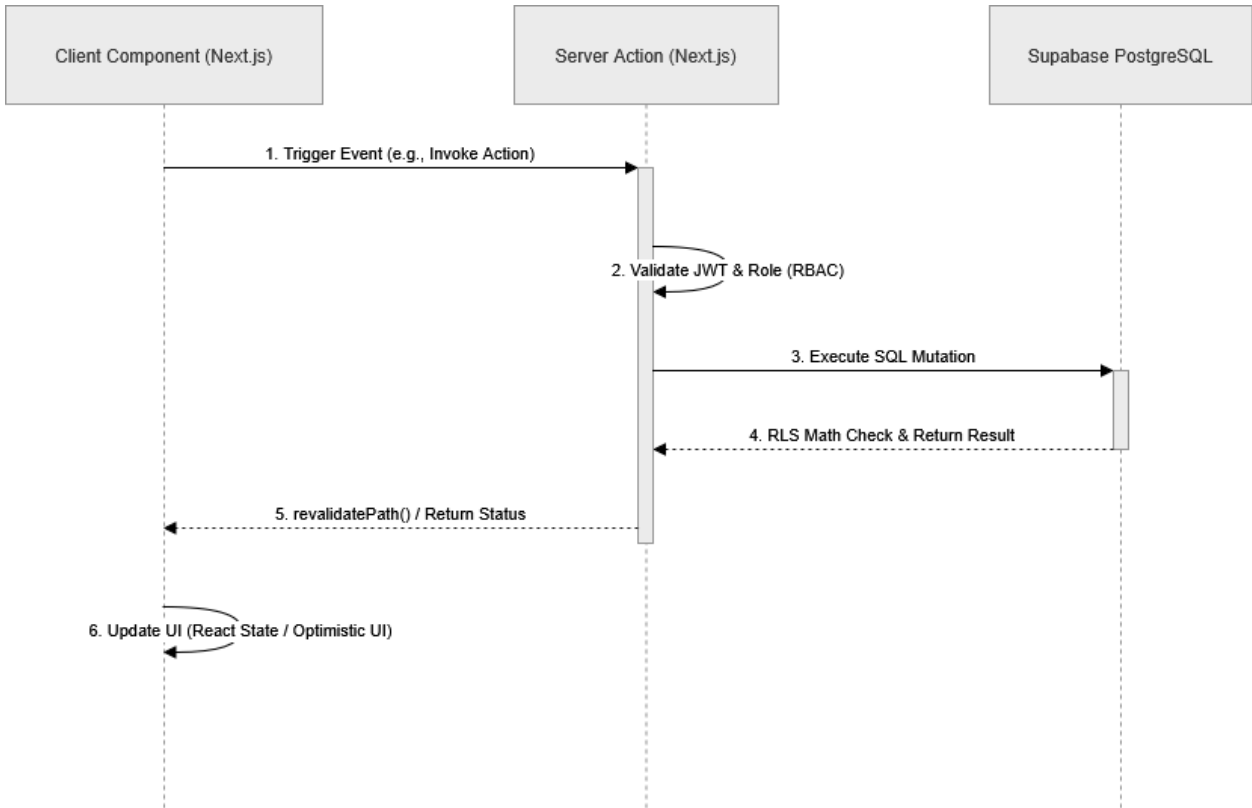
1. **Super Admin:** Platform-wide access; handles global accreditation and faculty adviser verification.
2. **Organization Officer (Tenant Admin):** Write-access scoped strictly to their specific org_id. They possess the ability to dynamically assign, edit, and filter sub-roles and capabilities (e.g., "Human Resources", "Organizer") for their members.
3. **Organization Member / Applicant:** Read-only or limited write-access (e.g., viewing internal feeds, utilizing QR digital IDs, or submitting applications).

Interface capabilities are conditionally rendered based on these roles, and all database mutations run through Next.js Server Actions which perform a secondary server-side role validation before execution

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA</i> v.0.01	

Data Flow & Integration

The system utilizes a streamlined, unidirectional data flow that bypasses traditional REST APIs for direct server-to-database communication.



State Management & Optimistic UI Updates

To ensure a highly responsive user experience, particularly for interactive organisms like the Kanban Board, Salina implements Optimistic UI updates. When a user initiates a state change (e.g., dragging an applicant card to a new pipeline stage), the Next.js frontend utilizes React's `useOptimistic` hook to instantly reflect the new state visually without waiting for network latency.

Simultaneously, the Next.js Server Action is fired asynchronously in the background. If the database mutation succeeds, `revalidatePath()` is called to sync the final server state. If the mutation fails (e.g., due to a network error), the optimistic state is automatically rolled back to its previous position, and the user is notified of the failure.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Error Handling & Logging Strategy

To ensure platform stability and a seamless user experience across all tenants, the system implements a multi-layered error handling and logging strategy:

- **Client-Side Boundaries:** The frontend utilizes Next.js `error.tsx` React Error Boundaries to catch unhandled runtime exceptions in Client Components. This prevents the entire application from crashing and displays a branded, user-friendly fallback UI with an option to recover or retry.
- **Server Action Catch Blocks:** All direct database mutations via Server Actions are wrapped in strict `try-catch` blocks. If a Supabase transaction fails (e.g., due to a Row Level Security violation, unique constraint error, or timeout), the server catches the exception and returns a standardized JSON error object to the client rather than throwing a raw 500 status code.
- **Form Validation:** Client-side and server-side inputs are validated using Zod schemas before executing database queries. This ensures that dynamic form submissions from the Policy Engine do not corrupt the database with malformed JSONB data.
- **Centralized Logging:** Critical backend failures and database timeout events are logged via the hosting platform's native edge logging (e.g., Vercel Logs) to allow developers to trace execution contexts and debug multi-tenant data fetching issues without exposing sensitive error stacks to the end-user.
- **Client-Side:** At the routing level, Next.js `error.tsx` boundary files are utilized to catch unexpected runtime errors and display graceful fallback UI components, preventing the entire application framework from crashing.
- **Server-Side:** All Supabase Server Actions are wrapped in strict `try/catch` blocks. If an action fails (e.g., due to an RLS violation or storage quota limit), the server returns a standardized, sanitized error object rather than exposing raw database logs. The frontend intercepts this object and triggers a non-intrusive toast notification to inform the user of the specific issue.

Deployment & Environments

The system infrastructure is managed through a streamlined CI/CD (Continuous Integration / Continuous Deployment) pipeline to safely push code from development to the cloud.

- **Hosting Infrastructure:** The Next.js frontend and Server Actions are deployed on Vercel, leveraging its Edge Network for low-latency server-side rendering and static asset delivery.
- **Database Hosting:** The PostgreSQL database, Auth, Storage, and Realtime services are hosted on a managed Supabase cloud instance.
- **Environment Tiering:**
 - **Local Development:** Developers run the Next.js client locally and utilize the Supabase CLI to spin up a local Dockerized instance of the database. This allows for safe schema modifications and RLS testing.
 - **Staging:** Pushes to the `main` branch trigger an automated preview deployment on Vercel, connected to a dedicated Supabase Staging project. This environment is used for QA testing and validating the Kanban UI and pipeline rendering.
 - **Production:** Approved releases are merged into the `production` branch, triggering a live deployment. Database changes are strictly applied using Supabase Migration files to ensure zero-downtime schema updates.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Design System & UI/UX (Foundational Elements)

Design Tokens (The Atoms)

Color Palette

Primary Color (C6623E)

Used for:

- Primary buttons
- Active States
- Navigation Highlights

Include:

- Primary Base(default interaction color)
- Primary Hover(darker variant)
- Primary Light(background tint)

Secondary Color (95CDE0)

Used for:

- Secondary actions
- Supporting UI accents

Accent Color (8A6ED4)

Used for:

- Tags, badges

Neutral Color (FBF5F2)

Used for:

- Used for layout structure, backgrounds, and readability

Typography

Typography is standardized and applied consistently across all modules:

- **H1** – Plus Jakarta Sans, Bold, 45px // (Page Titles)
- **H2** – Plus Jakarta Sans, Bold, 35px // (Section Headers)
- **H3** – Space Grotesk, Bold, 25px //(Card Titles)
- **H4** – Space Grotesk, Medium, 18px //(Subsections)
- **H5** – Space Grotesk, Regular, 16px // (Labels)
- **Paragraph** – Inter, Regular, 16px //(Body Text)

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Spacing and Grid

Desktop Grid Specifications

- Columns: 12
- Max Width: 1440px
- Gutter: 24px
- Margin: 80px

Desktop Spacing Scale

Based on a 4px system:

- 4px, 8px, 12px, 16px, 24px, 32px, 40px, 48px, 64px

Desktop Application in Wireframes

- Cards use 24px internal padding
- Sections separated by 32–48px
- Sidebar width consistent (~240–280px)

Iconography

Style: Outline-based, minimal

Stroke: 1.5–2px

Sizes:

- 16px (inline)
- 20px (buttons)
- 24px (navigation)

Used in:

- Sidebar navigation
- Table actions
- Status Indicators

Theming Atoms (Design Tokens)

Base Tokens: System-wide defaults (Grids, standard radii)

Border radius: 8px

Card shadow: subtle elevation

Grid system (fixed)

Spacing scale

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Variable Tokens: Properties that change per tenant (Primary brand color, Logo assets, Custom fonts).

These change depending on tenant configuration:

- Primary color
- Organization logo (AUTH screen and sidebar)
- Accent colors
- Theme(light/dark mode from Organization Table)

State Atoms: How components look during "Trial Expired" or "Read-only" states.

Disabled State

- triggered when an action is not allowed (invalid input, restricted function)

Properties:

- Reduced opacity (50%)No interaction

Read-only State - applied when users can view but not modify data

Properties:

- Inputs locked
- Buttons disabled or hidden

Component Library (Atoms & Molecules)

Atoms

Buttons

States:Default, Hover, Active, Disabled

Usage:

- AUTH(Login)
- System Admin (Save/Edit)
- Dashboards (Filters Actions)

Input Fields

States:Default, Focus, Disabled

Usage:

- Text input
- Password input(AUTH)
- Search input(dashboard)

Badges

Usage:

- Applicant Status
- Member Status
- Payment Status
- Verification Labels

Checkboxes

Usage:

- Form Input

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

- Table Selection

Molecules

Search bars (Input + Button)

Usage:

- Enables filtering of table data

Used in:

- Application Panel(AP)
- OF (Officer - Analytics)
- MB (Member - Management Board)

Card Headers

Usage:

- Analytic panels
- Dashboard cards

Form Groups

Usage: Label + input + validation message

Used in:

- AUTH(login form)
- System Admin (configuration forms)
- Dynamic Forms (Policy Engine)

Design Rationale & Accessibility

SaaS Experience (UX)

Onboarding Flow: The onboarding flow defines the initial interaction between a tenant (organization) and the platform. This flow is derived from the AUTH interface and system provisioning logic.

Onboarding Flow

- User logs in via AUTH interface
- Supabase Auth validates credentials
- JWT is issued with OrgID
- Tenant configuration is loaded
- User is redirected to dashboard

Dashboard Logic: How data visualizations aggregate tenant-specific metrics

- Cards display aggregated metrics
- Tables provide detailed records
- Search/filter refines datasets
- Modular layout supports scalability

Accessibility (a11y):

Compliance standards

- (WCAG 2.1)

Color Contrast Ratios

- Minimum 4.5:1 for text

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

- Background (#FBF5F2) ensures readability

Keyboard Navigation Support

- Full tab navigation
- Visible focus indicators
- Accessible form controls

Atomic System Design

Provisioning Organism (The Onboarding Engine)

Functions

- Validates organization accreditation
- Creates Organization record (OrgID)
- Assigns roles (Officer, Member)
- Seeds:
 - ☐ Recruitment policy
 - ☐ UI configuration
 - ☐ Initial Data

Subscription & Tiering (Feature Flagging)

SALINA uses a **single-tier access model**.

System Behavior

- Organization acquire access to the platform
- Once accredited and onboarded:
 - ☐ All features are fully available
 - ☐ No feature restrictions exist

Access Control

- Controlled via Role-Based Access Control (RBAC)

Role	Access
Super Admin	Full system access
Organization Officer	Full tenant access
Member / Applicant	Limited access

UI Behavior

- Components are shown/hidden based on role

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

- No feature locking based on payment tiers

Component Decomposition (The Organisms)

Organisms 1: Navigation Sidebar

Seen in: Member (MB), System Admin (SA)

Logic

- Route-based active state detection
- Dynamic rendering based on user role

Sub-components

- Logo
- Navigation items (icon + label)

Organism 2: Data Dashboard Grid

Seen in: Officer Dashboard (OF), Member (MB), AP

Structure

- Header: Search Bars, Filters
- Body: Metrics Cards, Tables, Charts

Data Fetching

- Next.js Server Actions
- Supabase queries
- Pagination (max 50 records)

State Management

- Optimistic UI updates
- React State Hooks

Templates & Page Layouts

Wireframes/Screen Images

- <https://drive.google.com/drive/folders/1wiqGUZYdcl6gKiavNXXchYn8WHbdIWFN?usp=sharing>

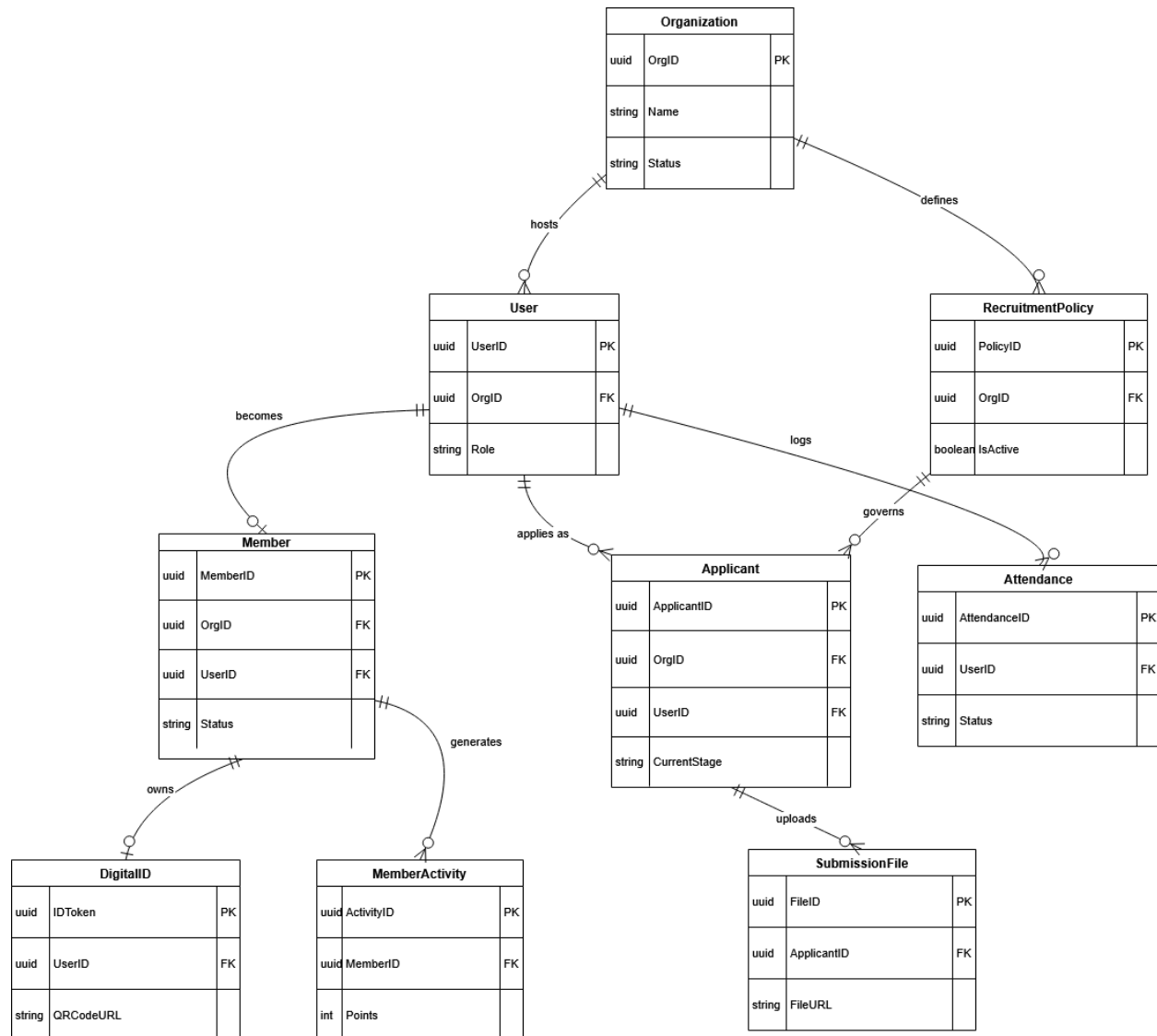
Data Design

Multi-tenant Schema Logic

Salina is designed as a multi-tenant Software-as-a-Service (SaaS) platform, serving as the centralized "Operating System" for organizations in general. To ensure data persistence and scalability, the system architecture utilizes a Supabase PostgreSQL backend paired with a Next.js client-server model.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA</i> v.0.01	

A foundational constraint of this architecture is strict tenant data isolation. Instead of relying solely on application-level filtering, data separation is mathematically enforced at the PostgreSQL database level using Row Level Security (RLS). This configuration allows Salina to utilize Server Actions for direct database communication—handling recruitment pipelines, membership tracking, and dynamic rosters—without requiring a traditional, separate middleware API layer.



Salina Entity-Relationship Diagram

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Global Platform Tables (Non-Tenant Scoped)

Table: Organization

Description: The core tenant table representing registered organizations on the platform.

Column Name	Data Type	Key	Description
OrgID	UUID	PK	Unique identifier for the organization.
Name	String		Official name of the organization.
Status	String		Operational status (e.g., Pending, Active).
PrimaryColor	String		Hex code for tenant branding.
FontStyle	String		Selected typography for the tenant portal.
Mode	String		Light or dark mode preference.
LogoURL	String		Hosted link to the organization's logo.
CreatedAt	Timestamp		Record creation date.
UpdatedAt	Timestamp		Last modification date.

Tenant-Scoped Tables (Enforced by RLS)

Table: User

Description: Represents authenticated individuals interacting with the platform.

Column Name	Data Type	Key	Description
UserID	UUID	PK	Unique identifier linked to Supabase Auth.
OrgID	UUID	FK	Links user to a specific organization tenant.
Email	String		User's registered email address.
FullName	String		User's complete name.
Role	String		RBAC assignment (e.g., Super Admin, Officer, Member).
IsVerified	Boolean		Indicates if the account email/identity is confirmed.
CreatedAt	Timestamp		Record creation date.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Table: Recruitment Policy

Description: Defines the dynamic workflow stages for an organization's application process.

Column Name	Data Type	Key	Description
PolicyID	UUID	PK	Unique identifier for the configuration.
OrgID	UUID	FK	The organization that owns this policy.
PipelineStages	JSONB / Array		Ordered list of stages (e.g., Interview, Deliberation).
IsActive	Boolean		Determines if this is the currently enforced policy.

Table: Applicant

Description: Tracks users actively moving through a recruitment pipeline.

Column Name	Data Type	Key	Description
ApplicantID	UUID	PK	Unique identifier for the application record.
OrgID	UUID	FK	The organization being applied to.
UserID	UUID	FK	The user submitting the application.
CurrentStage	String		Current location in the Kanban board.
ApplicationStatus	String		Overall state (e.g., Pending, Accepted, Rejected).
ApplicationData	JSONB		Dynamic form responses configured by the Policy Engine.
SubmittedAt	Timestamp		Date the application was finalized.

Table: Submission File

Description: Stores references to secure documents uploaded during the application process.

Column Name	Data Type	Key	Description
FileID	UUID	PK	Unique identifier for the uploaded file.
ApplicantID	UUID	FK	Links the file to a specific application record.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

StageName	String		The pipeline stage that required this file.
FileURL	String		Pointer to the Supabase Storage bucket location.
VerificationStatus	String		Status of officer review (e.g., Verified, Rejected).
VerifiedBy	UUID	FK	The Officer (User) who reviewed the document.
VerifiedAt	Timestamp		Timestamp of the review.

Table: Member

Description: Represents users who have successfully passed recruitment and are active in the roster.

Column Name	Data Type	Key	Description
MemberID	UUID	PK	Unique identifier for the active roster profile.
UserID	UUID	FK	The core user account linked to this profile.
OrgID	UUID	FK	The organization this member belongs to.
Status	String		Standing within the org (e.g., Active, Alumni, Probation).
DuesStatus	String		Financial standing (e.g., Paid, Unpaid).
Tags	Array		Custom categorization labels applied by Officers.
EngagementScore	Integer		Calculated metric based on member activity.

Table: Digital ID

Description: Manages the secure, scannable QR codes used for member verification and events.

Column Name	Data Type	Key	Description
IDToken	UUID	PK	Secure, unique token representing the ID.
UserID	UUID	FK	The user who owns this digital ID.
OrgID	UUID	FK	The organization issuing the ID.
QRCodeURL	String		Link to the generated visual QR code graphic.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

IsActive	Boolean		Determines if the ID can currently be scanned.
GeneratedAt	Timestamp		Date of initial ID creation.

Table: Attendance

Description: Logs instances of members checking into scheduled events.

Column Name	Data Type	Key	Description
AttendanceID	UUID	PK	Unique identifier for the check-in event.
EventID	UUID		Identifier for the specific event/meeting.
UserID	UUID	FK	The user who attended.
OrgID	UUID	FK	The organization hosting the event.
CheckInTime	Timestamp		Exact time the attendance was recorded.
CheckInMethod	String		How it was recorded (e.g., QR Scan, Manual Entry).

Table: Event

Description: Represents scheduled activities or meetings created by an organization that members can attend.

Column Name	Data Type	Key	Description
EventID	UUID	PK	Unique identifier for the event.
OrgID	UUID	FK	The organization hosting the event.
Name	String		The title of the event.
Description	Text		Detailed information about the event.
Location	String		Physical room or virtual link.
ScheduledDate	Timestamp		When the event is set to occur.
CreatedBy	UUID	FK	The Officer (User) who created the event.
CreatedAt	Timestamp		Record creation date.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Table: Member Activity

Description: Tracks granular interactions within the internal portal to calculate engagement.

Column Name	Data Type	Key	Description
ActivityID	UUID	PK	Unique identifier for the action.
MemberID	UUID	FK	The member performing the action.
PostID	UUID		The internal feed post interacted with.
ActivityType	String		Type of engagement (e.g., View, React, Comment, RSVP).
Points	Integer		Weighted value of the activity towards the engagement score.
RecordedAt	Timestamp		When the action occurred.

Because Salina allows multiple organizations to operate under a single shared backend, the database must be optimized for both strict security and high-volume query performance.

Security via Row Level Security (RLS): Authentication is managed through Supabase Auth using secure JSON Web Tokens (JWT). Every table containing tenant-specific data (e.g., Applicants, Members, Attendance) includes an **OrgID** foreign key. RLS policies are configured so that every database query automatically validates the user's JWT against the data's organizational owner. This guarantees that an officer or member can only query or mutate data belonging to their verified tenant.

Performance via Composite Indexing: To prevent massive sequential database scans as the platform grows, scoped tables utilize Composite Indexes where **OrgID** serves as the leading column. Because every tenant query is inherently filtered by their organization, leading-edge **OrgID** indexes ensure that features like the Kanban applicant tracking board and real-time internal feeds load instantly by isolating the query strictly to that tenant's partition. Additionally, GIN (Generalized Inverted Index) indexing is applied to JSONB columns, such as **ApplicationData**, to ensure the dynamic policy engine can filter custom form submissions efficiently.

Granular Role-Based RLS Policies: While the **OrgID** enforces baseline tenant isolation, granular RLS policies dictate role-based access control within that specific tenant. All database policies follow a "default-deny" architecture. Examples of explicit RLS rules include:

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

- **Applicant Table:**
 - *Select/Insert/Update*: Allowed if `auth.uid() = UserID` (Applicants managing their own submitted data).
 - *Select/Update*: Allowed if `auth.jwt() ->> 'org_id' = OrgID` AND the user possesses the **Officer** role (Officers managing the recruitment pipeline).
- **Organization Table:**
 - *Update*: Allowed only for the platform **Super Admin** or the specific tenant's verified **Officer**.

These mathematical checks guarantee that even if the Next.js client UI is manually bypassed or spoofed, unauthorized data mutation is blocked at the lowest infrastructure level.

API Design & Rate Limiting

Tenant Scoping

In a multi-tenant environment, the system must definitively know which organization's context an incoming request belongs to. While traditional REST APIs often utilize an **X-Tenant-ID** HTTP header, Salina abandons this approach due to the security risks of header spoofing. Instead, tenant scoping is securely enforced via cryptographic JWT claims.

Because Salina utilizes a Next.js App Router architecture with Server Actions, traditional API middleware is bypassed in favor of secure, direct server-to-database communication. The tenant identification flow operates as follows:

1. **JWT Custom Claims:** Upon successful authentication via Supabase Auth, a JSON Web Token (JWT) is generated. During this generation, a secure database trigger attaches the user's assigned **OrgID** into the token's payload as a custom claim.
2. **Session Context:** When a client triggers a Next.js Server Action (e.g., retrieving the applicant Kanban board), the server extracts this signed JWT from the secure session cookie.
3. **Database Execution:** The Next.js server passes the JWT directly to the Supabase PostgreSQL client. Because the JWT is cryptographically signed by the backend, the user cannot tamper with their assigned **OrgID**.
4. **RLS Enforcement:** Finally, the PostgreSQL database natively decodes the token during the transaction. Row Level Security (RLS) policies intercept the query and execute the mathematical check `auth.jwt() ->> 'org_id' = organization_id`, guaranteeing the query strictly interacts with the authorized tenant's data.

Quotas

In a shared, multi-tenant architecture, mitigating the "Noisy Neighbor" syndrome is critical to maintaining platform stability. If a single organization experiences a massive spike in traffic (e.g., during a major recruitment deadline) or executes poorly optimized bulk actions, it must not degrade the

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

performance of other tenants. Salina addresses this through a multi-tiered quota and rate-limiting strategy.

1. **Edge-Level API Rate Limiting (Compute)** To protect the database from connection exhaustion, rate limiting is enforced at the Next.js execution level (e.g., via Edge Middleware or within Server Actions) utilizing an in-memory data store like Redis.
 - **Tenant-Keyed Sliding Window:** Instead of merely rate-limiting by a user's IP address, the sliding window algorithm limits requests keyed by the **OrgID** extracted from the JWT. For example, an organization might be capped at 500 mutation requests per minute. If this threshold is breached, subsequent requests from that tenant receive a **429 Too Many Requests** status until the window resets, keeping the rest of the platform unaffected.
2. **Storage Quotas (Infrastructure)** Salina utilizes Supabase Storage to host applicant documents and receipts. To prevent a single tenant from exhausting the platform's storage bucket, a hard quota is enforced per organization.
 - **Enforcement Logic:** A **TotalStorageUsed** column is tracked per tenant. Before a Next.js Server Action initiates a secure upload to Supabase Storage, it checks this value against the tenant's allocated tier limit (e.g., 5GB). If the upload exceeds the quota, the transaction is rejected at the server level, preventing runaway storage costs.
3. **Query Complexity Caps (Database)** For resource-intensive operations—such as querying the GIN-indexed JSONB **ApplicationData** across thousands of applicants—pagination and strict limit clauses are mandatory. The platform UI does not permit unbounded **SELECT *** queries. All data grid endpoints enforce a maximum payload limit (e.g., 50 records per page) to ensure query execution times remain under acceptable latency thresholds.

Database Backups & Disaster Recovery

Because Salina operates on a single shared PostgreSQL database for all tenants, a centralized disaster recovery strategy is critical to ensure zero data loss in the event of hardware failure, malicious attacks, or accidental bulk deletions.

- **Automated Daily Backups:** The managed Supabase infrastructure automatically performs daily physical backups of the entire PostgreSQL database. These backups are retained securely off-site for a rolling period (e.g., 7 days on the standard tier) and can be used to restore the entire platform's state in a worst-case scenario.
- **Point-in-Time Recovery (PITR):** For the production environment, the database utilizes Write-Ahead Logs (WAL) to enable Point-in-Time Recovery. If a critical application error corrupts tenant data or an administrator executes a destructive bulk mutation by mistake, the database state can be rolled back to a specific minute before the incident occurred, minimizing tenant downtime and data loss.
- **Storage Redundancy:** Assets uploaded by applicants (such as identity documents and payment receipts) are hosted in Supabase Storage. These buckets are backed by standard cloud-provider redundancy across multiple availability zones to ensure high availability and protect against localized hardware failures.

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA v.0.01</i>	

Requirements Traceability Matrix

This Requirements Traceability Matrix (RTM) maps the functional requirements defined in the Software Requirements Specification (SRS) to the specific frontend and backend components of the system. Using the Atomic Design methodology, this matrix provides a clear breakdown of the components to be built and their corresponding verification methods.

Req ID	Component Name	Atomic Category	Verification Method
FR-1.1	AccreditationDataTable	Organism	Integration Test / UI Test
FR-1.2	GlobalMetricsDashboard	Template	End-to-End (E2E) Test
FR-1.3	AdviserApprovalModal	Molecule	Unit Test
FR-1.4	ThemeSettingsForm	Organism	Integration Test / UI Test
FR-2.1	PipelineConfigBuilder (JSON Engine)	Organism	Unit Test (Logic) / UI Test
FR-2.2	KanbanBoard & DraggableCard	Organism	E2E Test (Drag-and-Drop)
FR-2.3	DynamicApplicationForm	Organism	Integration Test
FR-3.1	MemberRosterGrid (with Tag Filters)	Organism	Integration Test
FR-3.2	PostComposer & EventScheduler	Molecule	Unit Test / UI Test
FR-3.3	QRScannerCamera	Molecule	Manual Test / E2E Test
FR-4.1	ApplicationStatusTracker	Molecule	Unit Test
FR-4.2	DigitalIDCard (QR Generator)	Molecule	Unit Test / UI Test
FR-4.3	NewsfeedStream (Supabase Realtime)	Organism	Integration Test
FR-4.4	MemberStandingDashboard	Organism	UI Test
FR-5.1	SupabaseAuthProvider (JWT)	Atom (Logic)	Integration Test
FR-5.2	PostgresRLSPolicies	Database Level	Automated Security/DB Test
FR-5.3	RBACServerActionWrapper	Atom (Logic)	Unit Test / Security Test

<i>SALINA</i>	Version: 0.01
Software Design Document	Date: 03/17/2026
<i>SALINA</i> v.0.01	

FR-5.4	SecureFileUploader (Supabase Storage)	Molecule	Integration Test
---------------	--	----------	------------------

Appendices

Design Tool Links:

Figma:<https://www.figma.com/design/TB7lQnxzBKCl0o93JqsnVt/SALINA?node-id=0-1&t=FyqmHaFWX9c456f4-1>